

**ECOLE SUPERIEURE D'INFORMATIQUE DE GESTION**



## Travail d'Algorithmes et Langage de Programmation Novembre 2025

### Informations générales

|                  |                                                      |
|------------------|------------------------------------------------------|
| <b>Branche :</b> | <b>ALGORITHMES ET LANGAGE DE PROGRAMMATION (ALP)</b> |
| <b>Classes :</b> | <b>ESIG 1 Classes A et B (groupes 1, 2 et 3)</b>     |
| <b>Date :</b>    | <b>28 octobre 2025</b>                               |

## Consignes générales

1. L'épreuve est **individuelle**.
2. **Documentation personnelle électronique** autorisée uniquement pour la partie 2, et ce, à condition qu'elle ait été copiée sur un emplacement local de la machine avant que l'épreuve ne débute. Aucune documentation n'est autorisée pour la partie 1.
3. La partie 1 dure 45 minutes au maximum et 15 minutes au moins. Aucune sortie aux toilettes n'est autorisée pendant la partie 1. Tout le temps restant après la reddition de la partie 1 pourra être mis à profit pour la partie 2.
4. Vous devez utiliser l'IDE **Thonny** pour développer vos programmes de la partie 2. Aucun autre environnement de développement (IDE) n'est autorisé.
5. **Toute communication externe est strictement interdite.** Cela inclut :
  - a. Le partage de ressources via le réseau,
  - b. Toute tentative de contact avec un humain ou un outil d'intelligence artificielle. Cela comprend également l'installation et l'utilisation d'extensions ou de plugins dans les IDE qui utilisent l'intelligence artificielle pour générer des suggestions de code en temps réel,
  - c. La présence de dispositifs connectés à proximité immédiate (téléphone portable, montre connectée, lunettes intelligentes, clé USB ou tout autre support de stockage ou de communication). *Vous devez déposer ces objets à l'endroit indiqué par le surveillant.*

⚠ Tout manquement à ces règles sera considéré comme une fraude et sanctionné par la note minimale.

## Préparation et installation

### Avant le début de l'épreuve

1. **Connectez-vous au réseau** sur le poste de travail qui vous a été attribué.
2. **Copiez votre documentation personnelle** (clé USB, Google Drive, etc.) vers un emplacement local, par exemple dans un répertoire à votre nom dans `C:\ESIGUsers`
3. **Copiez l'archive de l'épreuve**
  - a. Ouvrez le répertoire `Lanschool Files` situé sur le Bureau.
  - b. Déplacer l'archive `ALP-Nov25.7z` à la racine de `C:\ESIGUsers`.  
*Cette archive est protégée par un mot de passe qui vous sera communiqué au démarrage de l'épreuve.*
4. **Eteignez l'écran** en attendant de recevoir l'énoncé de la partie 1. L'écran pourra être rallumé pour la partie 2.

### Une fois le mot de passe reçu et la partie 1 rendue

1. **Décompressez l'archive** pour obtenir le répertoire suivant : `C:\ESIGUsers\ALP-Nov25`
2. Commencez la partie 2 **en indiquant vos nom et prénom** dans un commentaire en en-tête des fichiers modifiés

⚠ Assurez-vous de **travailler sur les fichiers extraits** et non sur l'archive compressée

## Soumission du travail

1. **Sauvegardez votre travail et fermez votre IDE (Thonny)** avant toute autre action
2. **Renommez** le dossier `ALP-Nov25` situé dans `C:\ESIGUsers` en y ajoutant vos **nom et prénom** à la fin de façon à avoir un dossier `ALP-Nov25-NomPrenom`.
3. **Créez une archive du dossier renommé**
  - a. Faites un clic droit sur le dossier renommé.
  - b. Sélectionnez "Compresser vers" > "Fichier 7z".
4. **Effectuez le rendu de votre travail**
  - a. Double-cliquez sur le raccourci "Rendu ALP Nov25" présent dans le dossier `ALP-Nov25-NomPrenom`.
  - b. Déposez l'archive `.7z` précédemment créée dans le répertoire de rendu à votre nom.

⚠ **Seul le travail déposé dans votre répertoire de rendu sera évalué.** Il est de votre responsabilité de vérifier que tous les fichiers nécessaires y sont bien présents.

⚠ **Soyez ponctuel lors de votre reddition** afin d'éviter une éventuelle pénalité.

## Consignes générales

### Sur l'organisation

1. Lisez tous les documents fournis.
2. Complétez les différentes questions de manière qu'elles répondent aux spécifications de l'énoncé.

### Sur la programmation en Python

1. Interdiction d'utiliser les fonctions prédéfinies `max`, `min`, `index`, `count`, `sum`, `mean`, `filter` ou équivalent dans des modules fournis par python.
2. Les éléments suivants sont explicitement autorisés : la fonction `len` et le mot clé `in` uniquement dans le contexte d'une boucle `for` (`for ... in ...`).
3. Pas de résultats ou de données codés « en dur » : votre code doit pouvoir s'adapter à d'autres jeux de données.
4. Les seules variables globales autorisées sont les constantes.
5. L'affichage produit par vos programmes doit correspondre exactement aux affichages attendus.

## Partie 1 : Débranchée

Nom étudiant-e : .....

Prénom étudiant-e : .....

Enseignant : .....



Si vous détachez les feuilles de l'énoncé Partie 1 :  
écrivez **sur chaque page** vos **nom et prénom**.

**Aucune** documentation n'est autorisée pour cette partie

*L'écran de votre ordinateur doit être **éteint***

## Question – Lire du code

Indiquez avec exactitude et précision ce qu’affiche la console pour chacun des extraits de code ci-dessous.

Q1

```
def f1(v1):  
    v1 = v1 + f2(v1)  
    print('f1', v1)  
  
def f2(v2):  
    print('f2', v2)  
    return v2+1  
  
v1 = 1  
f1(v1)  
print(v1)
```

Output de la console :

Q2

```
def a(c):  
    return b(2, c)  
  
def b(c, d):  
    print('b', c, d)  
    z = 0  
    if c < d:  
        e(c+d)  
    else:  
        e(c-d)  
    return z - d  
  
def e(z):  
    z += 1  
    print('e', z)  
  
print('1-1')  
print(a(1))  
print('1-2')  
print(a(2))  
print('1-3')  
print(a(3))
```

Output de la console :

Q3

```
nombres = [3, 4, 5, 5]

c = 0
for i in range(len(nombres)):
    j = 0
    while j < len(nombres) and c < 2:
        print(nombres[i], nombres[j], c)
        if nombres[i] == nombres[j]:
            c += 1
        j += 1
    print('$')
```

Output de la console :

## Partie 2 : Programmation Python



La **documentation personnelle est autorisée** pour cette partie.

**N'oubliez pas de sauvegarder** régulièrement votre travail.

Assurez-vous de **travailler sur les fichiers extraits**

## Question 1 : Suite de figures géométriques

Votre travail s'effectuera dans le fichier `question1.py`.

Un magasin de jouets pour enfants souhaite proposer, durant la période hivernale, une activité ludique permettant aux clients de dessiner une suite de figures au moyen d'un programme informatique.

Le programme doit être capable de tracer deux figures de base : un carré (quatre côtés égaux, quatre angles droits) et un triangle équilatéral (trois côtés égaux, trois angles de  $60^\circ$ ). Deux procédures distinctes doivent être définies pour dessiner ces figures, chacune recevant en paramètre la longueur du côté.

Une procédure supplémentaire doit ensuite dessiner l'ensemble des figures, disposées les unes à la suite des autres sur une ligne, en appelant les deux procédures créées précédemment. Cette procédure reçoit en paramètres un **périmètre maximum**, une **longueur de côté initiale** ainsi qu'un **pourcentage d'augmentation**. Tous les 3 tours de boucle, la figure tracée est un carré ; aux autres tours, il s'agit d'un triangle équilatéral. Après chaque tracé, la tortue se positionne à la droite de la figure dessinée pour préparer la suivante. La longueur du côté de chaque nouvelle figure est augmentée du pourcentage indiqué par rapport à la précédente. Le dessin s'arrête dès que le périmètre total cumulé des figures dépasse la limite fournie.

Le périmètre d'une figure correspond à la somme des longueurs de ses côtés. Par exemple, un carré de côté 50 a un périmètre de  $4 \times 50 = 200$ , et un triangle équilatéral de côté 50 a un périmètre de  $3 \times 50 = 150$ .

Le programme demande à l'utilisateur les trois valeurs d'entrée nécessaires pour dessiner la suite de figures, puis la dessine. Toutes les données saisies par l'utilisateur sont supposées correctes.

Une procédure `initialiser_tortue()` est déjà fournie dans le fichier. Elle permet de régler la vitesse d'exécution, de cacher le curseur en forme de tortue, et de positionner le point de départ du dessin plus à gauche de l'écran. Vous devez l'appeler après la récupération des données d'entrée et avant de commencer à dessiner votre suite de figures.

### Résultats attendus :

**1** = Périmètre maximum    **2** = Longueur de côté initiale    **3** = Pourcentage d'augmentation

| <b>1</b> | <b>2</b> | <b>3</b> | Suite de figures dessinées |
|----------|----------|----------|----------------------------|
| 900      | 50       | 0        |                            |
| 600      | 15       | 50       |                            |
| 1500     | 20       | 10       |                            |

## Question 2 : Mission sauvetage – gestion des vivres

Votre travail s'effectuera dans le fichier `question2.py`.

Vous êtes les programmeurs chargés de gérer une flotte de bateaux de sauvetage qui transporte de la nourriture et des médicaments vers un pays se situant aux abords de la Méditerranée.

Dans un premier temps, vous écrivez un programme qui a pour but d'évaluer l'approvisionnement en vivres nécessaire à l'équipage d'un bateau.

Votre programme doit lire des données, les valider, puis calculer la quantité totale d'eau et de nourriture à embarquer et afficher un récapitulatif.

Veillez à diviser votre code en fonctions et procédures et ajouter des constantes quand c'est approprié ; séparez calculs et affichage.

Entrées (dans cet ordre)

1. `n`: nombre de personnes à bord (entier > 0)
2. `jours`: durée du trajet en jours (entier > 0)
3. `climat`: un mot parmi "froid", "moyen", "chaud"
4. `vulnérables`: "oui" ou "non" (indique s'il y a des personnes vulnérables à bord)

Règles de calcul

- Base par personne et par jour:
  - eau = 3.0 L
  - nourriture = 0.75 kg
- Ajustements liés au climat:
  - En climat chaud, ajouter 40% d'eau.
  - En climat froid, ajouter 20% de nourriture.
  - En climat moyen, pas de modification
- Ajustement groupes vulnérables:
  - S'il y a des personnes vulnérables à bord et que le voyage dure 5 jours ou plus, ajouter encore 10% d'eau et 10% de nourriture à la valeur qui tient déjà compte du climat.

Validations à effectuer à l'aide d'une fonction de validation booléenne

- Refuser  $n \leq 0$  ou  $jours \leq 0$  (message d'erreur clair).
- Refuser tout climat différent de "froid", "moyen", "chaud".
- Refuser toute valeur de vulnérables différente de "oui" ou "non".

Sortie attendue

- Afficher les totaux et les ajustements appliqués.
- En cas d'erreurs, n'effectuez pas le calcul et affichez un message explicite pour chaque erreur rencontrée

**Exemples d'exécution:**

Nb personnes:1  
Durée:1  
Climat:moyen  
Vulnérables?oui  
Il faut 3.0l d'eau et 0.75kg de nourriture.  
Ajouts 'climat'. Eau: 0l - Nourriture: 0kg  
Ajouts groupes vulnérables. Eau: 0l - Nourriture: 0kg

Nb personnes:10  
Durée:5  
Climat:chaud  
Vulnérables?oui  
Il faut 231.0l d'eau et 41.25kg de nourriture.  
Ajouts 'climat'. Eau: 60.0l - Nourriture: 0kg  
Ajouts groupes vulnérables. Eau: 21.0l - Nourriture: 3.75kg

Nb personnes:50  
Durée:10  
Climat:froid  
Vulnérables?non  
Il faut 1500.0l d'eau et 450.0kg de nourriture.  
Ajouts 'climat'. Eau: 0l - Nourriture: 75.0kg  
Ajouts groupes vulnérables. Eau: 0l - Nourriture: 0kg

Nb personnes:50  
Durée:10  
Climat:froid  
Vulnérables?non  
Il faut 1500.0l d'eau et 450.0kg de nourriture.  
Ajouts 'climat'. Eau: 0l - Nourriture: 75.0kg  
Ajouts groupes vulnérables. Eau: 0l - Nourriture: 0kg

Nb personnes:0  
Durée:0  
Climat:free  
Vulnérables?humans  
Nb personnes doit être > 0  
Durée doit être > 0  
Climat inconnu  
Vulnérables : "oui" ou "non"

## Question 3 : Mission sauvetage – gestion des drones

Votre travail s'effectuera dans le fichier `question3.py`. Veillez à diviser votre code en fonctions et procédures et ajouter des constantes quand c'est approprié.

Dans la même mission de sauvetage, afin de garantir la sécurité des membres de l'équipage, des drones ont été mis à disposition pour accompagner les bateaux dans leur mission difficile.

Votre programme gère 3 listes :

1. La liste des noms de bateaux, elle contient 11 bateaux
2. La liste du nombre de personnes à bord de chacun des 11 bateaux
3. La liste de pourcentage de batterie actuel de chaque drone accompagnant les bateaux correspondants.

Les 3 listes sont parallèles, c'est-à-dire que les indices correspondent entre chaque liste.

Exemple: le bateau `Hind` a un équipage de 5 personnes, et est suivi par le drone ayant 64.2 % de batterie.

Afin d'assurer le bon fonctionnement de cette mission de sauvetage, vous êtes chargé.e.s de :

1. Calculer et afficher le nombre total de membres de la flottille
2. Afficher tous les noms de bateaux dont le drone a une charge supérieure ou égale à la moyenne des batteries des drones
3. Calculer combien de personnes pourront d'ores et déjà partir avec l'accompagnement des drones. Les bateaux peuvent partir uniquement si la batterie du drone qui les accompagne est supérieure ou égale à 70%. Afficher le message correspondant.
4. Récapituler le niveau de batterie du drone qui accompagne le Karma (index 6). En effet, leur bateau a peur du mauvais karma et voudrait être préparé à charger la batterie dès que c'est nécessaire. Sachant que le trajet dure 8 jours, et que la batterie est consommée de 15% par jour, proposer un récapitulatif de pourcentage de batterie au fil des jours et avertir lorsque la recharge devient nécessaire (batterie en dessous de 10%). Dans le cas où la recharge n'a pas été nécessaire, votre code doit afficher un message correspondant (ex. « la recharge n'est pas nécessaire»). Le code doit fonctionner pour n'importe quelle durée de mission.

### Sortie console attendue :

Afficher les résultats sous le format qui est présenté dans l'output ci-dessous.

**Question 3 : Exemple d'exécution**

----- Rapport du jour de la flottille -----

Question 1 : Nombre total de membres dans la flottille : 64

Question 2 : Bateaux dont le drone a une batterie  $\geq$  à la moyenne (53.25%) :

- Hind(64.3%)
- Aurora(88.5%)
- Al Khalil(55.1%)
- Free Willy(72.2%)
- Sirius(71.2%)
- Wahoo(83.3%)
- Yaffa(95.0%)

Question 3 : Nombre d'équipiers pouvant déjà partir (drones  $\geq$  70%) :  
27 sur 64

Question 4 : Récapitulatif des chargements nécessaires pour le drone accompagnant Karma

Jour 0 bateau Karma situation drone 25.0

Jour 1 bateau Karma situation drone 21.25

Jour 2 bateau Karma situation drone 18.0625

Jour 3 bateau Karma situation drone 15.353125

Jour 4 bateau Karma situation drone 13.05015625

Jour 5 bateau Karma situation drone 11.0926328125

Pour le bateau Karma la recharge est nécessaire au jour 6